Reactive Following with Five Robots: A Five-Rule Controller in Webots and ROS 2

Lara Dehandschutter
FEUP / Universidade do Porto
up202603432@up.pt

Lennert De Rouck
FEUP / Universidade do Porto
up**XXXXXXXXX**@up.pt

Pedro Cancela
FEUP / Universidade do Porto
up202400230@up.pt

***Abstract*—We present a purely reactive controller that makes five differential-drive robots form a following chain in Webots, driven by five ROS 2 nodes. Each robot acts on a single current snapshot from a frontal camera and LiDAR (100°, 4 m); there is no target memory, no global pose and no inter-robot communication. Robot orientation is read directly from colour: the front half of each chassis is red and the rear half green, so "facing me" and "facing away" reduce to which colour is nearer. The controller is five prioritised rules. The robots assemble into a stable train — one leader on its base curve and four followers — in roughly 3 min from a scattered start, and hold it thereafter without further avoidance events. We report following duration, angular error and target losses, and devote the larger part of this paper to the quirks and failure modes that remain, several of which are consequences of the sensing envelope rather than of the control rules.**

I. Introduction

The task is reactive following: five robots, each deciding only from its own current sensor readings, should aggregate into one or more following chains. The constraints are those of classical reactive control in the sense of Brooks' subsumption architecture [1]: behaviour is a priority ordering over situation–action rules, with no world model and no planning. The assignment additionally forbids GPS, global localisation, target memory and privileged state, while permitting internal controller state of the PI/PID kind.

Our contribution is deliberately small. The interesting content of this report is not the controller, which is roughly 120 lines, but the catalogue of ways in which a correct-looking reactive rule set misbehaves once five copies of it interact in a shared environment. Section VI is therefore the longest section, by intent.

II. Simulation Model and Sensing

A. Robot

Each robot is a differential-drive cylinder of radius 0.115 m with two powered wheels and an invisible contact bumper of radius 0.146 m. Two robots are in contact when their centres are 0.292 m apart; this number is the reference against which all proximity results below are read.

B. Colour-coded orientation

An earlier iteration used two small recognition plaques, one red at the front and one green at the rear. These were replaced by two coloured half-shells covering the front and rear halves of the chassis. The reason is angular coverage: a flat plaque shrinks to nothing in projection as a robot turns away, and we measured that the leader became unclassifiable once the angle between its heading and the observer's line of sight exceeded about 74° — precisely when a leader was turning, which is when a follower most needs to keep it.

Each half is a separate Webots `Solid` with its own `recognitionColors`. One detail is load-bearing: Webots reports the *origin* of a recognised Solid, so the two halves are offset to $x = \pm 0.060\,\mathrm{m}$. Sharing the chassis centre would make both halves report the same position and front/back would be indistinguishable.

C. Sensors

Sensing is frontal only, as required: a recognition camera and a LiDAR, both 100° and 4 m. The LiDAR mounting deserves comment because it caused most of our early failures and its fix is non-obvious.

Mounted centrally on the roof at $z = 0.155\,\mathrm{m}$ — the natural place, and necessary to clear its own chassis — the LiDAR sits *above* the 0.125 m body top of every other robot. With a narrow vertical field of view, the beam only descends to body height at

$$d \;=\; \frac{0.155 - 0.125}{\tan(\theta_v/2)} \;\approx\; 0.75\,\mathrm{m}, \tag{1}$$

so a peer closer than 0.75 m is invisible. Every collision threshold we might wish to set lies inside that blind radius. Worse, the blindness is unavoidable for a *central* sensor: to avoid seeing its own rim at radius r_b the beam drop at r_b must stay under the clearance c, while seeing a peer at distance d requires the drop at $d - r_b$ to reach c. Both θ_v and c cancel, leaving $d > 2r_b$ — twice the body radius, which is exactly the distance at which two robots already touch.

The resolution exploits the fact that the LiDAR is now frontal. Mounted at the *front edge* ($x = 0.130\,\mathrm{m}$, outside the body radius), every forward ray's closest approach to the chassis lies behind the sensor, so it can be lowered to $z = 0.115\,\mathrm{m}$: inside the 0.045 m to 0.125 m band of every peer. Robots are then visible at all ranges including contact. A 360° LiDAR cannot do this, because its rearward rays would face its own chassis.

TABLE I
RULES, IN EVALUATION ORDER. FIRST MATCH WINS.

#	State	Condition	Action
1	`ESCAPE_*`	red nearer, $d_r < 0.45$	escape
2	`AVOID_RED`	red nearer, $d_r < 0.90$	turn right
3	`FOLLOW`	green visible	steer to it, speed ramp
4	`APPROACH`	red only, $d_r \geq 0.90$	steer to it
5	`OBSTACLE_*`	no robot, LiDAR < 2.0	escape
–	`BASE_CURVE`	otherwise	v_0, ω_{base}

TABLE II
PER-ROBOT RESULTS, 291.2 S RUN.

	Following	%	Losses	Mean $\lvert\beta\rvert$ (rad)
R1	290.9 s	99.9	2	0.629
R2	291.1 s	99.9	1	0.702
R3	44.7 s	15.3	2	0.420
R4	282.8 s	97.1	3	0.729
R5	137.2 s	47.1	6	0.451

D. Software architecture

Webots runs one controller per robot, which packs a camera/LiDAR snapshot into a UDP datagram. A ROS 2 bridge republishes each as `/Ri/local_sensors`; five independent `ReactiveFollower` nodes subscribe and publish `/Ri/cmd_vel`. No node subscribes to another robot's topics, which is what makes the no-communication constraint auditable rather than merely asserted.

III. Reactive Controller

Perception reduces each frame to the nearest red and the nearest green, as (distance, bearing) pairs. The controller is five rules in priority order (Table I).

The *escape* shared by rules 1 and 5 reverses at 0.06 m s^{-1} if the LiDAR reads under 0.25 m (`_BACK`), and otherwise rotates on the spot, always clockwise, until the forward cone is clear (`_TURN`). It never advances. Being stateless, it simply stops firing once the way ahead opens, and whichever rule applies next drives off in the direction the rotation happened to reach.

Three design points proved necessary rather than cosmetic.

a) Red is a comparison, not a detection: Rules 1–2 require red to be *nearer than green*, not merely visible. Every robot being followed presents a red half somewhere in view, so keying on visibility made followers abandon correct leaders.

b) Rules 1 and 5 must agree on a direction: They respond to the same physical situation seen through different sensors, so they must not disagree about which way to turn. Section VI-A describes what happened when they did.

c) Turn rate is capped by radius, not by a fixed rate: Steering straight at a leader is pure pursuit, which cuts inside the leader's path. In a five-robot ring the leader sits about 0.63 rad off-axis, giving $\omega = 1.5 \times 0.63 = 0.94\,\text{rad}\,\text{s}^{-1}$ and an arc radius of $v/\omega \approx 0.15\,\text{m}$ against a 0.95 m ring: the formation spirals inward until it piles up. We cap by $\omega_{\max} = v_0/R_{\min}$ rather than by a shared ω, because the robots run at different speeds (0.12 m s^{-1} to 0.16 m s^{-1}) and a shared cap would give each a different radius and distort the ring.

IV. Experimental Method

The five robots start evenly spaced on a 0.95 m circle, each heading along the tangent. We record per robot: following duration, mean absolute bearing error while following, target losses (transitions out of a following state) and bumper contacts. A seeded script (`tools/randomize_starts.py`) regenerates scattered starting poses so that runs can be repeated with different positions and headings while remaining reproducible, and a command-line flag selects the turning strength for the no-following / weak / strong comparison. Variance across starting configurations has not yet been quantified, so the timings below should be read as characteristic rather than as means over a sample.

V. Results

Table II gives per-robot figures from a 291 s observation of the controller described above. The system is quiet: 48 mode changes across all five robots, 9.9 min^{-1}, where earlier iterations of this controller produced several hundred per minute.

Time-weighted occupancy was `FOLLOW` 71.9%, `BASE_CURVE` 27.5%, and all avoidance states together below 1% (`AVOID_RED` 0.3%, `ESCAPE_TURN` 0.2%, `AVOID_WALL` 0.1%).

Formation proceeds in five phases: 82 s of scattered search; 34 s of first acquisitions; an 11 s congestion incident; 59 s of re-forming; and then a stable train, which persisted for the remaining 105 s with no avoidance event at all and showed no sign of degrading. The end state is the intended one — R3 leading on its base curve, R1, R2, R4 and R5 following — which is why R3's low following percentage is a success rather than a failure: the leader of a train is, by definition, not following anything.

Mean absolute bearing error while following was 0.549 rad (31.5°). This should not be read as tracking error. In a five-robot ring the leader is geometrically about 36° off-axis, so the measured value is close to the best achievable for this formation.

VI. Quirks, Shortcomings and Limitations

Each item below states the mechanism, the evidence for it, what it costs in behaviour, and what removing it would require. Several are properties of the sensing envelope and cannot be fixed inside the control rules at all.

A. Close-range escape: two rules that disagreed

The original design had rule 1 turn clockwise and rule 5 turn toward whichever half of the cone was more open. The two then contradicted each other whenever a robot was close enough that the camera could not reliably classify it.

At contact the bearing to a peer is extremely sensitive to lateral offset: at 0.15 m, 1 cm sideways is 6° and 8 cm is

39°. Two robots pressed together therefore put each other's red half near the $\pm 50°$ cone edge, where detection alternates frame to frame. Logs show the consequence directly: of 53 `AVOID_WALL` commands, none carried any camera detection, and they interleave with `ESCAPE_TURN` lines reading `red=0.15@+0.86`. The pair alternated between a clockwise command and an anticlockwise one and sat spinning against each other, faces touching.

The same rule also made a robot grind along a wall, for two further reasons: it returned full forward speed while turning, so it drove into what it was avoiding; and clearance was measured over only $\pm 15°$, so a wall met obliquely was never "ahead" at all and nothing fired.

Both are now one escape, shared by rules 1 and 5, always clockwise, never advancing, reversing when the LiDAR reads under 0.25 m, and measuring clearance over $\pm 45°$. We stopped trying to distinguish the cases: a peer seen only by its wheel is indistinguishable from a wall, and since the correct response is the same, the distinction was never needed. The remaining weakness is that the escape is purely open-loop in direction — it always turns the same way, which is what guarantees two robots separate, but also means a robot in a corner will rotate until some direction happens to clear rather than choosing the best one.

B. Bistable detection at the edge of the field of view

A target sitting near $\pm 50°$ is alternately inside and outside the camera cone from one frame to the next, because the bearing fluctuates by more than the margin separating it from the boundary. The controller has no notion of a marginal detection: a target is either present or absent, and different rules fire in the two cases.

The congestion incident shows the consequence. R3 and R5 each met an oncoming robot, turned right, and recovered in about 1.4 s — the rule behaving exactly as intended. R4 instead alternated between `AVOID_WALL` and `ESCAPE_TURN` three times in 4 s. Those two rules require contradictory inputs: rule 1 needs a red robot inside 0.45 m, rule 5 needs *no* robot visible at all. Both were true in alternate frames because the robot was at $+49.2°$, which is the bearing recorded when the episode finally resolved. The robot was not confused about what to do; it was given two different worlds to act in.

The cost is a few seconds of thrashing, and then however long the train takes to re-form around the disturbed robot — of the order of a minute. Suppressing it requires asymmetric acquire/drop thresholds — accept a target at $50°$ but only discard it past, say, $52°$ — which is a deliberate piece of remembered state and therefore a judgement call against the brief's prohibition on target memory. We left it in rather than quietly adding hysteresis.

C. The highest-priority rule is not the fastest-acting rule

All commands pass through an acceleration limiter before reaching the wheels, which exists so that a target appearing at the edge of the cone cannot produce an instantaneous full-rate swing. It is applied uniformly, including to the emergency rule.

`ESCAPE_TURN` commands $v = 0,\ \omega = -0.80$. The log records $v = 0.12,\ \omega = +0.37$ on the tick it engages: the robot is still rolling *forward* and still turning *left*, because the limiter is unwinding whatever the previous rule had commanded. Full reversal of a turn takes of the order of 0.2 s, during which the highest-priority rule in the system is only partially in effect. The robot travels perhaps 0.02 m toward the hazard in that window — small, but it means the priority ordering in Table I describes the *decision*, not the actuation. A correct treatment would exempt rule 1 from the limiter, at the cost of permitting one genuinely abrupt command.

D. Nothing can be sensed to the side or rear

Both sensors span $100°$ forward. A robot alongside or behind is not poorly sensed, it is *unsensed*: no reading distinguishes it from empty space. Sideways and rear-end contact are therefore undetectable in principle, and no control rule can respond to them, because no rule has an input that changes when they occur.

Everything we do about it is preventive rather than reactive — the turn-rate cap stops followers cutting across each other's paths, and the priority of rule 3 keeps a follower locked to the robot ahead rather than swinging toward a newcomer. These reduce the frequency of robots ending up abreast; they cannot detect or recover from it. This is the single largest gap between the system and a deployable follower, and closing it needs a sensor, not a rule: either a rear camera, a side-mounted range sensor, or staggering the LiDAR heights per robot so peers remain mutually visible through a full circle.

E. Formation geometry is coupled to the turn cap by hand

Three parameters that look independent are not. For five robots on a ring, spacing and radius satisfy $s = 2R\sin(\pi/5) = 1.176R$, and the camera reads 0.19 m less than centre spacing because it sits 0.130 m ahead of its own centre and the green half 0.060 m behind the leader's. A chosen standoff therefore *implies* a ring radius, which must in turn be something the turn-rate cap can actually drive.

We violated this without noticing. A standoff of 0.45 m implies a 0.54 m ring, while the cap refused any arc tighter than 0.90 m. The two demands are unsatisfiable simultaneously, and the observable result was that all five robots closed to the standoff, hit the speed floor and crawled at $0.03\,\mathrm{m\,s^{-1}}$ — a symptom that looks like a control bug and is actually an inconsistent parameter set. The present values agree, but nothing in the code checks the relationship, and anyone retuning one of the three will reintroduce the fault. A constructor assertion would catch it.

F. The controller cannot be single-stepped

Snapshot freshness is judged with a wall-clock timer, while the simulator emits snapshots on simulation time. The two agree only while Webots runs at approximately real time. Stepping the simulation tick by tick — the natural way to inspect a manoeuvre — halts simulation time while wall time

continues, so every robot declares its data stale and drops to `SENSOR_FAILSAFE` within a second of each click.

This is not merely an inconvenience: it means the system cannot be observed in the mode one would most want to observe it in, and a log recorded while stepping is not comparable with one recorded while running. We produced exactly that confusion, attributing 70 % of robot-time to sensor dropouts and concluding the simulator was running below real time; the same configuration run freely spends 0.0 % in that state. The conclusion was wrong and the evidence had looked strong. A simulation-time clock would remove the problem.

G. Logging is event-driven, so within-state extremes are invisible

A line is written only when a robot changes mode. A quantity that rises and falls *within* one mode leaves no trace. This is not a cosmetic issue: it caused us to misdiagnose a collision. `APPROACH` was recorded at $\omega = 0.27$, and the next line — a different mode — showed 0.73. Since the rule that follows caps ω at 0.14, the 0.73 must have been carried in, so the true peak inside `APPROACH` was at least 0.73 and plausibly the 1.1 ceiling. The log showed a mild turn immediately before a violent one.

Target range has since been added to each line, which helps, but the sampling remains event-driven. Any statistic computed from these logs understates within-state extremes, and the metrics in Section V should be read with that in mind.

H. The delivered controller has no test coverage

The repository contains ten test modules. All of them exercise the previous `safe_logic` cascade and its marker-based perception, neither of which is part of the delivered system; they import symbols that no longer exist and would fail at collection. The current controller — the one that produced every result in this paper — is covered by no automated test at all.

This matters more than it looks, because the suite was green throughout the period when the LiDAR could not see a robot closer than 0.75 m. The tests were written against synthetic scans and agreed with every change we made, including changes that did nothing in simulation. Passing tests were an active source of false confidence, and we came to rely on measurements from the simulator instead.

VII. Conclusion

A five-rule reactive controller, acting on one frontal snapshot at a time, assembled five robots into a stable leader-plus-four-followers train in 186 s and held it without further intervention. The rules were cheap; the difficulty was almost entirely in the interaction between the sensing envelope and the geometry of a multi-robot formation. The dominant failure mode we did not eliminate — bistable detection at the edge of the field of view — is a property of where the sensor stops, not of what the controller decides.

References

[1] R. A. Brooks, “A robust layered control system for a mobile robot,” *IEEE J. Robot. Autom.*, vol. 2, no. 1, pp. 14–23, 1986.

[2] R. C. Arkin, *Behavior-Based Robotics*. Cambridge, MA, USA: MIT Press, 1998.

[3] O. Michel, “Webots: Professional mobile robot simulation,” *Int. J. Adv. Robot. Syst.*, vol. 1, no. 1, pp. 39–42, 2004.

[4] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, 2022.

[5] R. C. Coulter, “Implementation of the pure pursuit path tracking algorithm,” Robotics Institute, Carnegie Mellon Univ., Tech. Rep. CMU-RI-TR-92-01, 1992.